

Hash Function: Hash Function and its Application

Chapter-7: Cryptographic Data Integrity Algorithms

Dr. Bhasha Anjaria

Assistant Professor

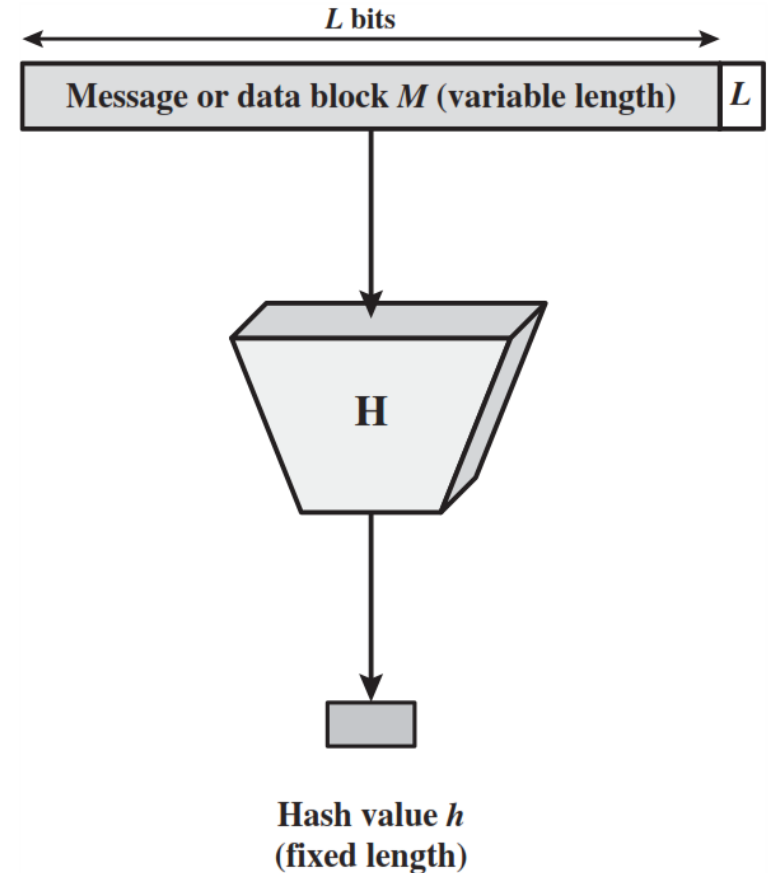
Department of Computer Science and Engineering

Content

1. Hash Function: Hash Function and its Application
2. Security Requirements for Cryptographic Hash Functions
3. Hash Functions Based on Cipher BlockChaining
4. Secure Hash Algorithm (SHA).
5. MAC: Message Authentication Requirements
6. Message Authentication Functions
7. Requirements for Message Authentication Codes
8. Security of MACs
9. HMAC
10. Digital Signature: Introduction to Digital Signatures
11. Digital Signature Standard

Hash Functions

- A hash function H accepts a variable-length block of data M as input and produces a fixed-size hash value $h = H(M)$.
- A “good” hash function has the property that the results of applying a change to any bit or bits in M results, with high probability, in a change to the hash code.

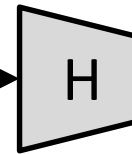


Hash Functions

Message

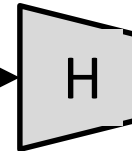
Message
digest

Alice was beginning to get very tired of sitting by her sister on the bank, and have nothing to do.



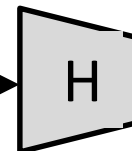
DFDC349A

I am not a crook



FB93E283

I am not a cook



A3F4439B

Hash Functions Applications

- Message authentication
- Digital Signature
- One-way password file

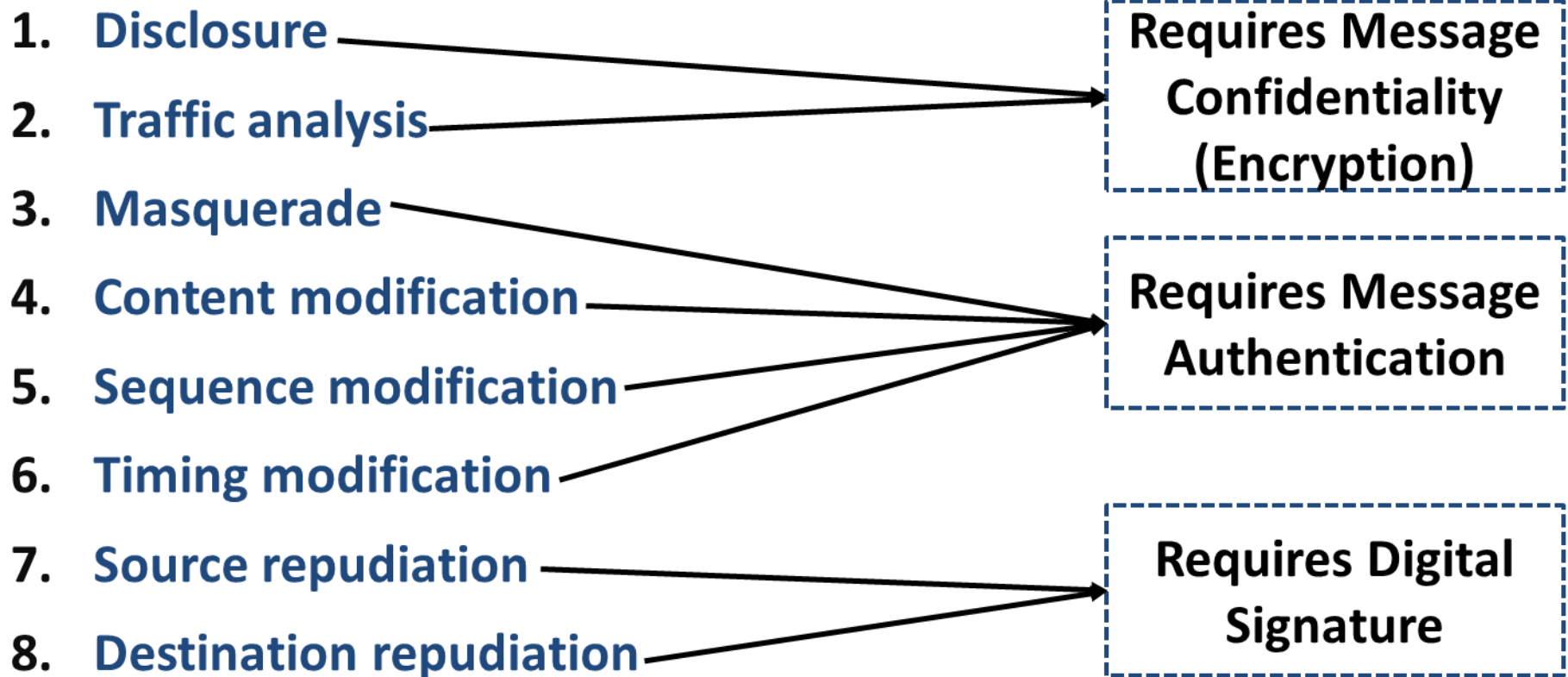
Security Requirement for Cryptographic Hash Functions

- Disclosure
- Traffic analysis
- Masquerade
- Content modification
- Sequence modification
- Timing modification
- Source repudiation
- Destination repudiation

Security Requirement for Cryptographic Hash Functions

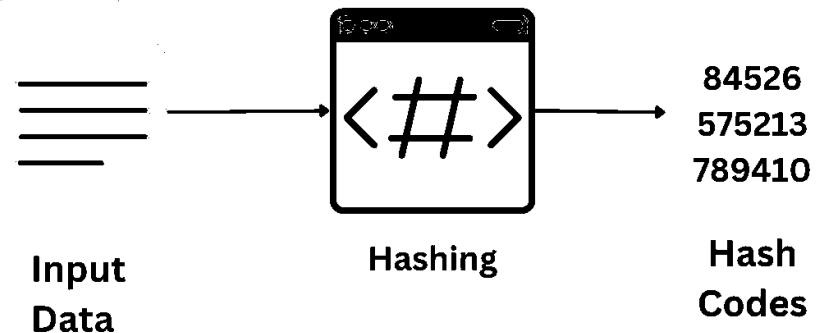
- **Disclosure:** Release of message contents.
- **Traffic analysis:** Analyses the traffic and observe the pattern of traffic between parties.
- **Masquerade:** Insertion of messages into the network from a fraudulent source.
- **Content modification:** Changes to the contents of a message.
- **Sequence modification:** Any modification to a sequence of messages between parties.
- **Timing modification:** Delay or replay of messages.
- **Source repudiation:** Denial of transmission of message by source.
- **Destination repudiation:** Denial of receipt of message by destination.

Security Requirement for Cryptographic Hash Functions



Hash Functions Based on Cipher Block Chaining

- A hash function maps data of arbitrary size to a fixed-size string. Used for data integrity, digital signatures, password storage, etc.
- Essential properties:
 - Deterministic
 - Fast computation
 - Pre-image resistance
 - Collision resistance
 - Avalanche effect



Hash Functions Based on Cipher Block Chaining

- CBC is a mode of operation for block ciphers.
- Each plaintext block is XORed with the previous ciphertext block before encryption.
- Requires Initialization Vector (IV) for the first block.
- Common in encryption standards like AES.

Hash Functions Based on Cipher Block Chaining

- CBC-MAC (Message Authentication Code) uses a block cipher in CBC mode to produce a fixed-size output.
- Treats input message as blocks. Only the final ciphertext block is used as the MAC.
- **Steps:**
 - Split message into blocks.
 - XOR each block with the previous ciphertext.
 - Encrypt using a symmetric cipher (e.g., AES). Use last output as the MAC/hash.

Hash Functions Based on Cipher Block Chaining

- **Advantages:**

- Simple to implement using existing block ciphers.
- Efficient for fixed-length messages.

- **Limitations:**

- Insecure for variable-length messages without modification.
- Requires key management (not ideal for general-purpose hashing).

- **Fixes:**

- Add padding, message length, or use key diversification.

Secure Hash Algorithm (SHA) – Requirement of Hash Functions

1. Can be applied to any **length of message M**.
2. Produces **fixed-length output h**.
3. It is **easy to compute $h=H(M)$** for any message M.
4. Given hash value **h** is **infeasible** to find **y** such that **$(H(y) = h)$**
One-way property (In other words, given a fingerprint, we cannot derive a matching message).
5. For given block **x**, it is computationally infeasible to find
 $y \neq x$ with $H(y) = H(x)$
Weak collision resistance
6. It is computationally infeasible to find messages **m1** and **m2** with
 $H(m1) = H(m2)$
Strong collision resistance

Secure Hash Algorithm (SHA) – Simple Hash Functions

- The input (message, file, etc.) is viewed as a sequence of n-bit blocks.
- The input is processed one block at a time in an iterative fashion to produce an n-bit hash function.
- One of the simplest hash functions is the bit-by-bit exclusive-OR (XOR) of every block.

$$C_i = b_{i1} \oplus b_{i2} \oplus \dots \oplus b_{im}$$

Where,

C_i = ith bit of the hash code $1 \leq i \leq n$

m = number of n-bit blocks in the input

b_{ij} = ith bit in jth block

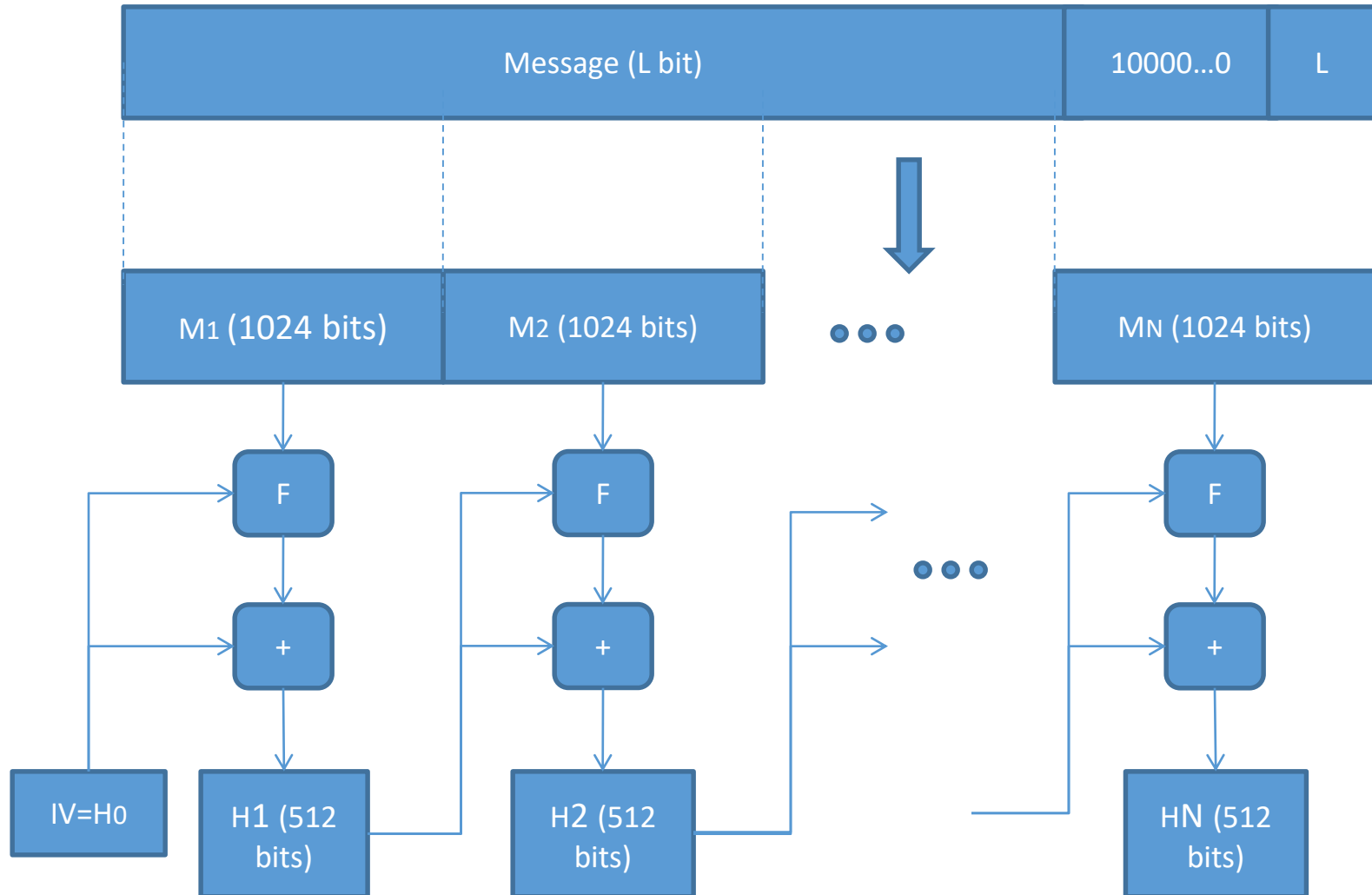
Secure Hash Algorithm (SHA)

	SHA - 1	SHA - 224	SHA - 256	SHA - 384	SHA - 512
Message Digest Size	160	224	256	384	512
Message Size	< 264	< 264	< 264	< 2128	< 2128
Block Size	512	512	512	1024	1024
Word Size	32	32	32	64	64
Number of Steps	80	64	64	80	80

Secure Hash Algorithm (SHA) - 512

- The algorithm takes input as a message with a maximum length of less than 2^{128} bits and produces output as a **512-bit** message digest.
- The input is processed in **1024-bit** blocks.

Secure Hash Algorithm (SHA) - 512



Secure Hash Algorithm (SHA) – 512 (Step 1 – Append Padding Bits)

- The message is padded so that its length is congruent to **896 modulo 1024** [$\text{length} \equiv 896(\text{mod } 1024)$].
- Padding is always added, even if the message is already of the desired length.
- Thus, the number of padding bits is in the range of **1 to 1024**.
- The padding consists of a single 1 bit followed by the necessary number of 0 bits.

Secure Hash Algorithm (SHA) – 512 (Step 2 – Append Length)

- A block of 128 bits is appended to the message.
- This block is treated as an unsigned 128-bit integer (most significant byte first) and contains the **length of the original message** (before the padding).

Secure Hash Algorithm (SHA) – 512 (Step 1 and 2 Example)

- Example:
- Given is the message “abc” consisting of three 8-bit ASCII characters with a total length of $l = 24$ bits.

A	B	C
01100001	01100010	01100011

- We append a “1” followed by $k = 871$ zero bits, where k is determined by
- $k \equiv 896 - (l + 1) = 896 - 25 = 871 \pmod{1024}$.
- Finally, we append the 128-bit value which contains the binary representation of the length $l = 24$ $10 = 110002$.
- The padded message is then given by

A	B	C	Append	Length
01100001	01100010	01100011	10000..0	00..011000

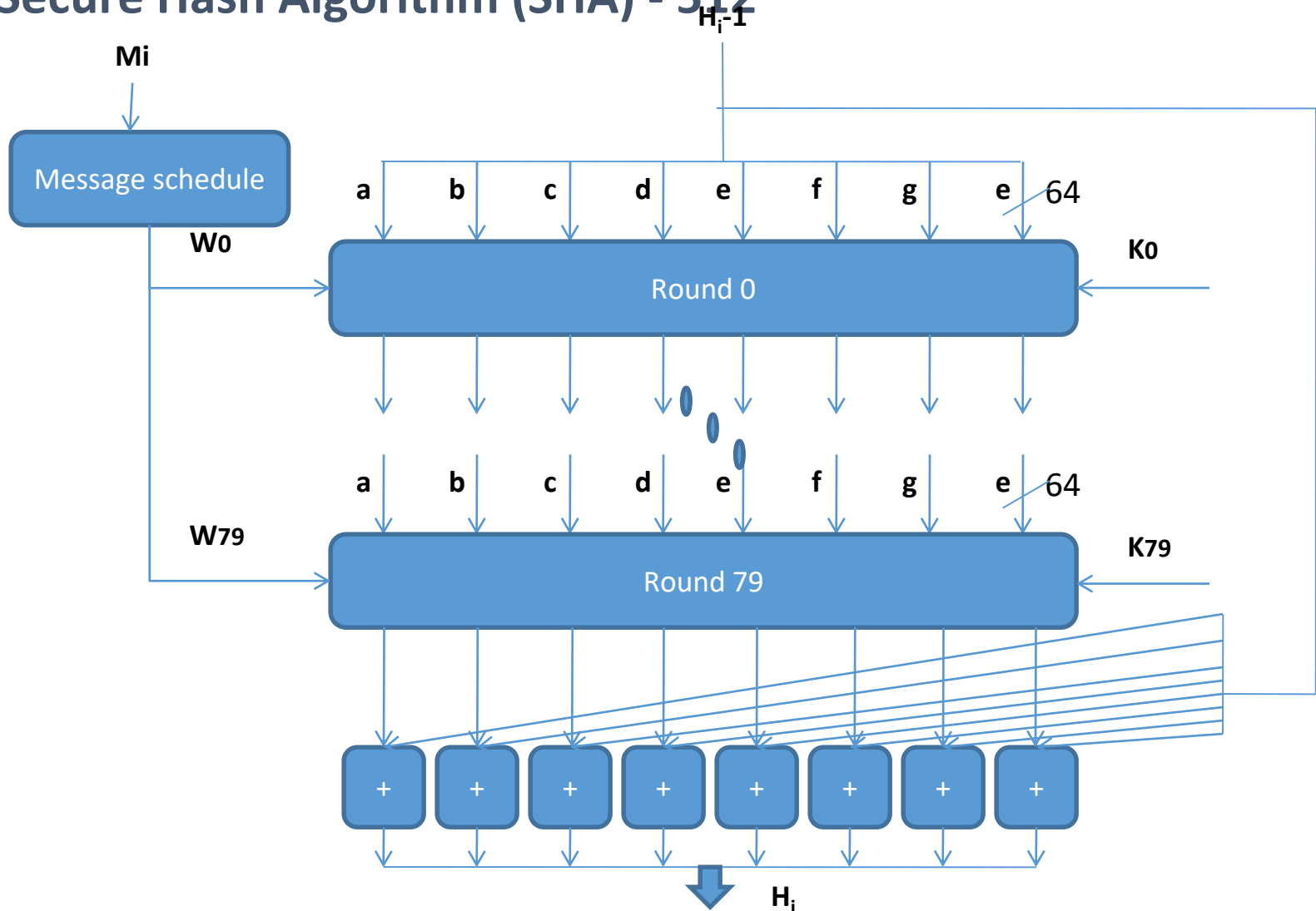
Secure Hash Algorithm (SHA) – 512 (Step 3 – Initialize Hash Buffer)

- The outcome of the first two steps produces a message that is an integer multiple of 1024 bits in length.
- The expanded message is represented as the sequence of 1024-bit blocks $M_1, M_2, \dots, M_N$, so that the total length of expanded message is $N \times 1024$ bits.
- A 512-bit buffer is used to hold intermediate and final results of the hash function. The buffer can be represented as **eight 64-bit registers** (a, b, c, d, e, f, g, h).

Secure Hash Algorithm (SHA) – 512 (Step 4 – Process message in 1024 bits)

- The heart of the algorithm is a module that consists of 80 rounds and this module is labelled as F function.

Secure Hash Algorithm (SHA) - 512



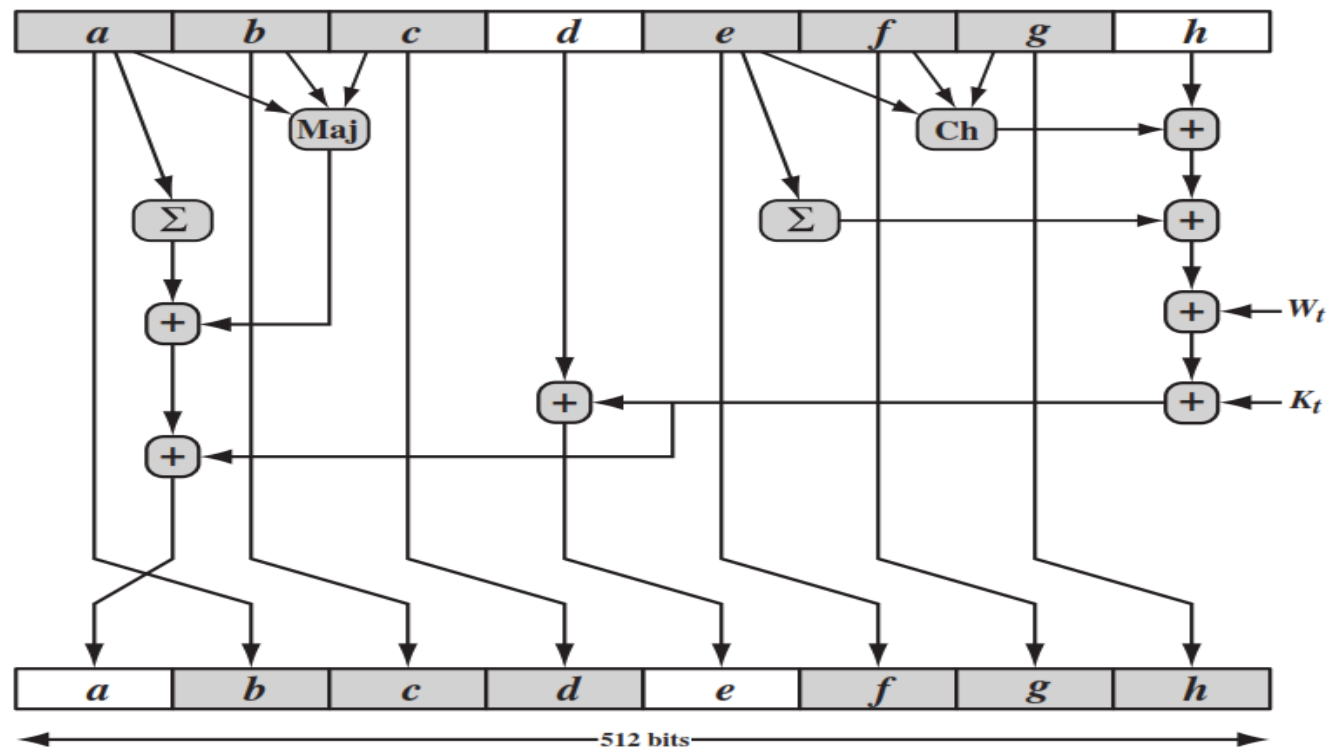
Secure Hash Algorithm (SHA) - 512

- Each round takes as input the 512-bit buffer value, abcdefgh, and updates the contents of the buffer.
- At input to the first round, the buffer has the value of the intermediate hash value, H_{i-1} .
- Each round t makes use of a 64-bit value W_t , derived from the current 1024-bit block being processed.
- The output of the eightieth round is added to the input to the first round (H_{i-1}) to produce H_i .

Secure Hash Algorithm (SHA) - 512

- After all **Nth block of 1024-bit** have been processed, the output from the **Nth stage is the 512-bit** message digest.

Secure Hash Algorithm (SHA) - 512



$$\begin{aligned} h &= g \\ g &= f \\ f &= e \\ e &= d + T_1 \end{aligned}$$

$$\begin{aligned} d &= c \\ c &= b \\ b &= a \\ a &= T_1 + T_2 \end{aligned}$$

$$T_1 = h + \text{Ch}(e, f, g) + \left(\sum_1^{512} e\right) + W_t + K_t$$

$$T_2 = \left(\sum_0^{512} a\right) + \text{Maj}(a, b, c)$$

Secure Hash Algorithm (SHA) - 512

- $\text{Maj}(a,b,c) = (a \text{ AND } b) \text{ XOR } (b \text{ AND } c) \text{ XOR } (a \text{ AND } c)$ Majority of arguments are true.
- Conditional function = $(e \text{ AND } f) \text{ XOR } (\text{NOT } e \text{ and } g)$
- $\Sigma(a) = \text{ROTR}(a,28) \text{ XOR } \text{ROTR}(a,34) \text{ XOR } \text{ROTR}(a,39)$
- $\Sigma(e) = \text{ROTR}(e,14) \text{ XOR } \text{ROTR}(e,18) \text{ XOR } \text{ROTR}(e,41)$
- $+$ = addition modulo 2^{64}
- K_t = a 64-bit additive constant.
- W_t = a 64-bit word derived from plaintext.
- ROTR = Circular right shift rotation

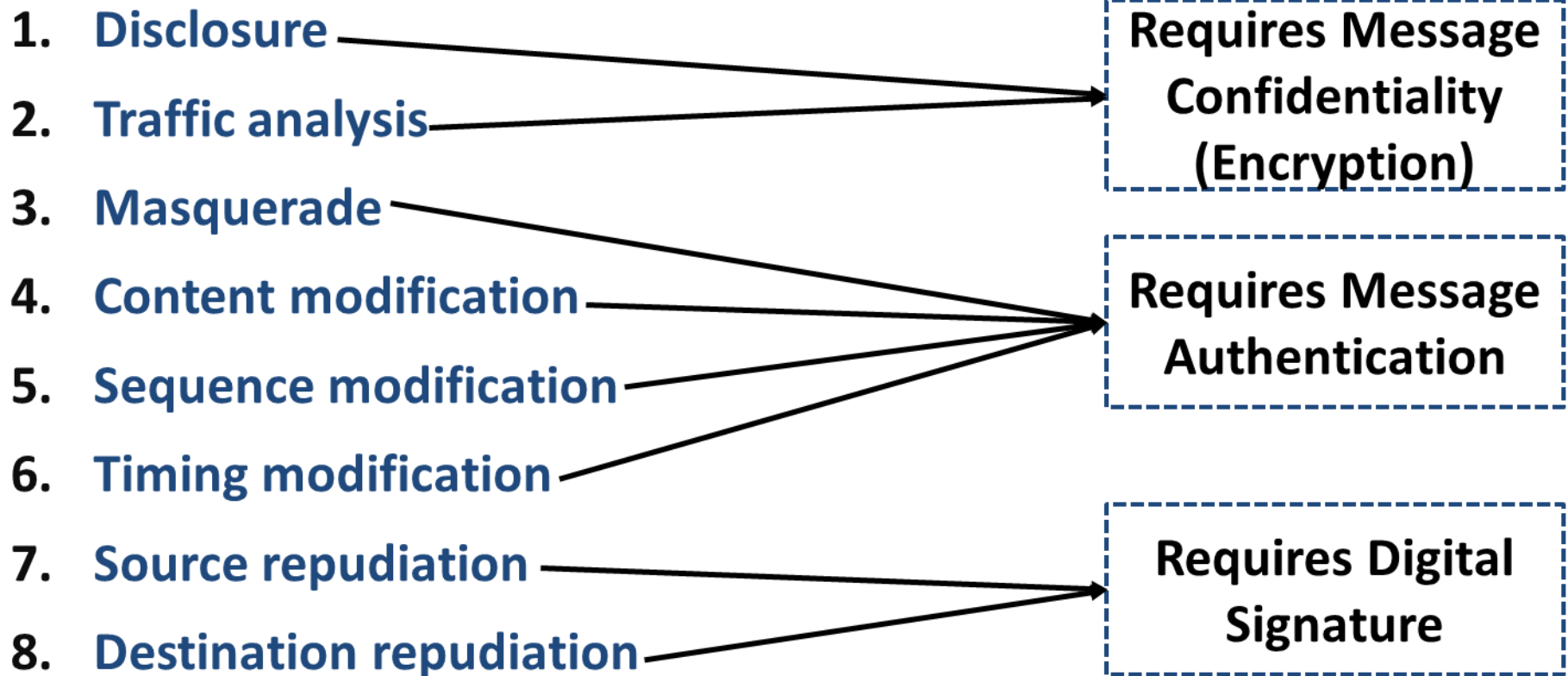
MAC Message Authentication Requirement

- Disclosure
- Traffic analysis
- Masquerade
- Content modification
- Sequence modification
- Timing modification
- Source repudiation
- Destination repudiation

MAC Message Authentication Requirement

- **Disclosure:** Release of message contents.
- **Traffic analysis:** Analyses the traffic and observe the pattern of traffic between parties.
- **Masquerade:** Insertion of messages into the network from a fraudulent source.
- **Content modification:** Changes to the contents of a message.
- **Sequence modification:** Any modification to a sequence of messages between parties.
- **Timing modification:** Delay or replay of messages.
- **Source repudiation:** Denial of transmission of message by source.
- **Destination repudiation:** Denial of receipt of message by destination.

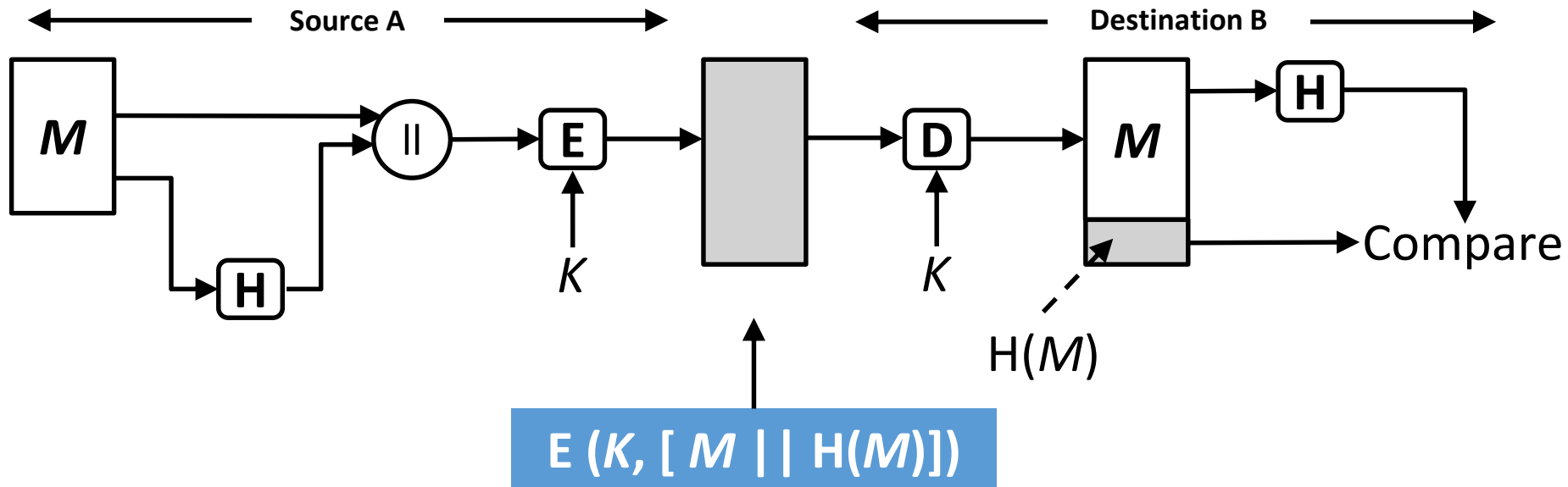
MAC Message Authentication Requirement



Message Authentication Functions

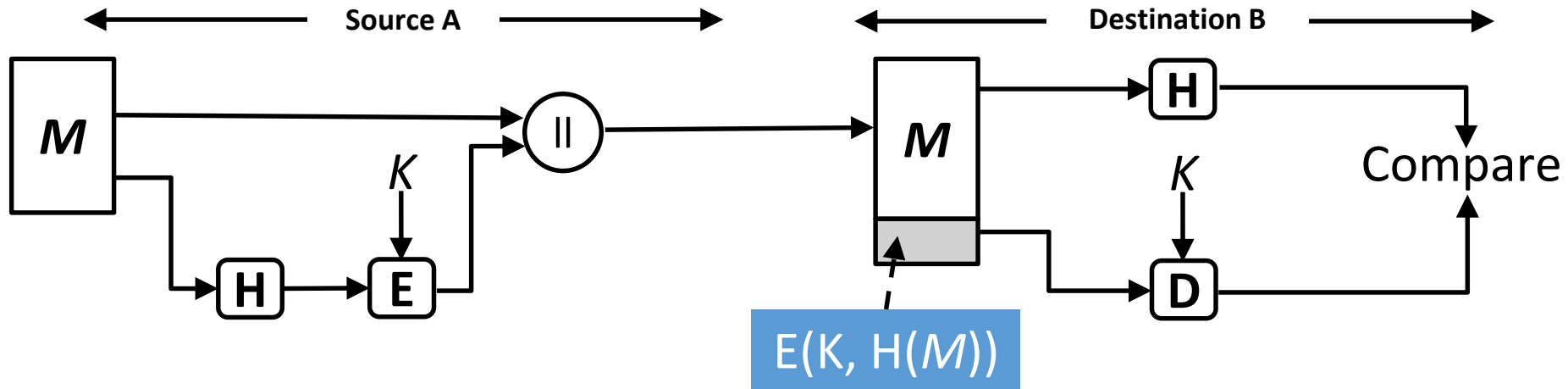
- Message authentication is a mechanism or service used to verify the integrity of a message.
- Message authentication assures that data received are exactly as sent (i.e., contain no modification, insertion, deletion, or replay).
- When a hash function is used to provide message authentication, the hash function value is often referred to as a message digest.

Message Authentication Functions



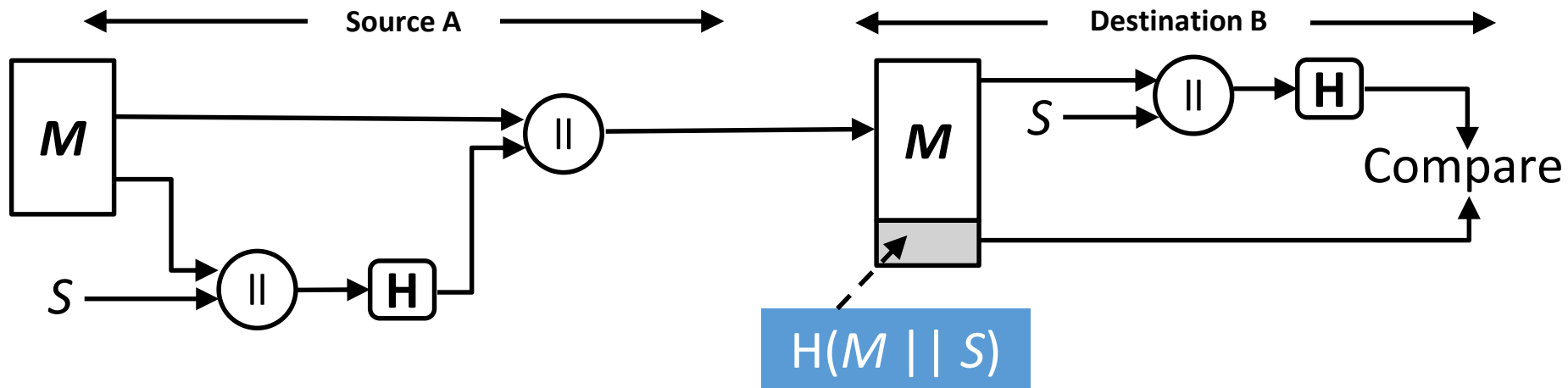
- Only A and B share the secret key, the message must have come from A and has not been altered.
- The hash code provides the structure required to achieve authentication.
- Because encryption is applied to the entire message plus hash code, confidentiality is also provided.

Message Authentication Functions



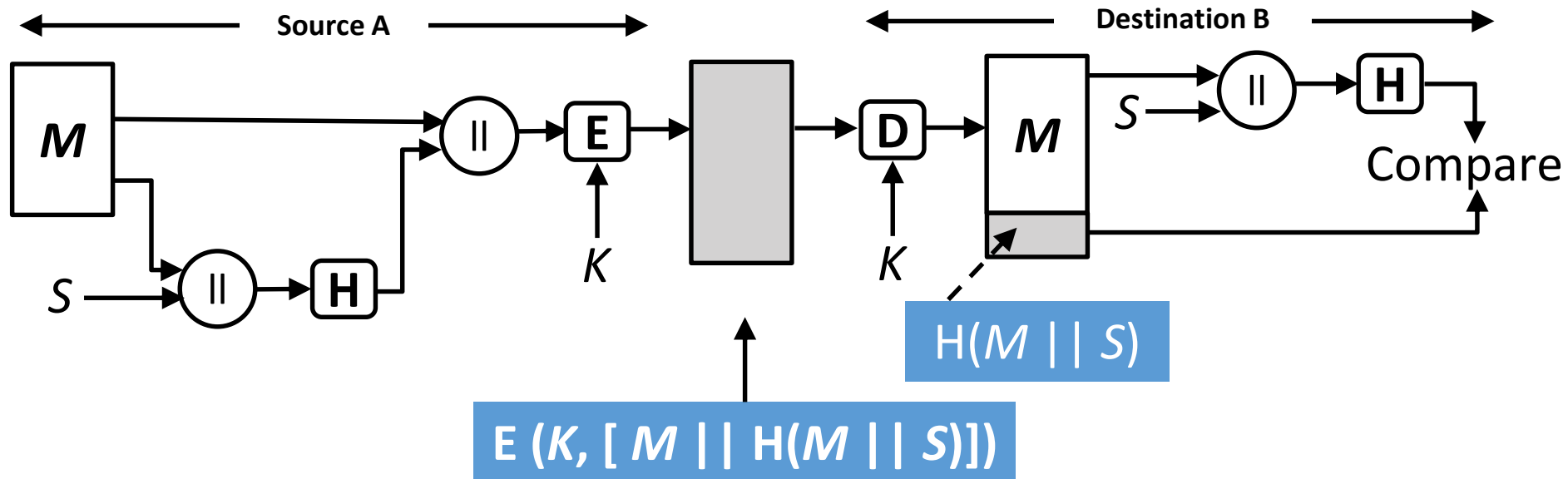
- Only the hash code is encrypted, using symmetric encryption.
- This reduces the processing burden for those applications that do not require confidentiality.

Message Authentication Functions



- It is possible to use a hash function but no encryption for message authentication.
- A and B share a common secret value S .
- A computes the hash value over the concatenation of M and S and appends the resulting hash value to M . Because B possesses S , it can recompute the hash value to verify the message.
- An opponent cannot modify an intercepted message.

Message Authentication Functions



- Confidentiality can be added to the approach of method (3) by encrypting the entire message plus the hash code.

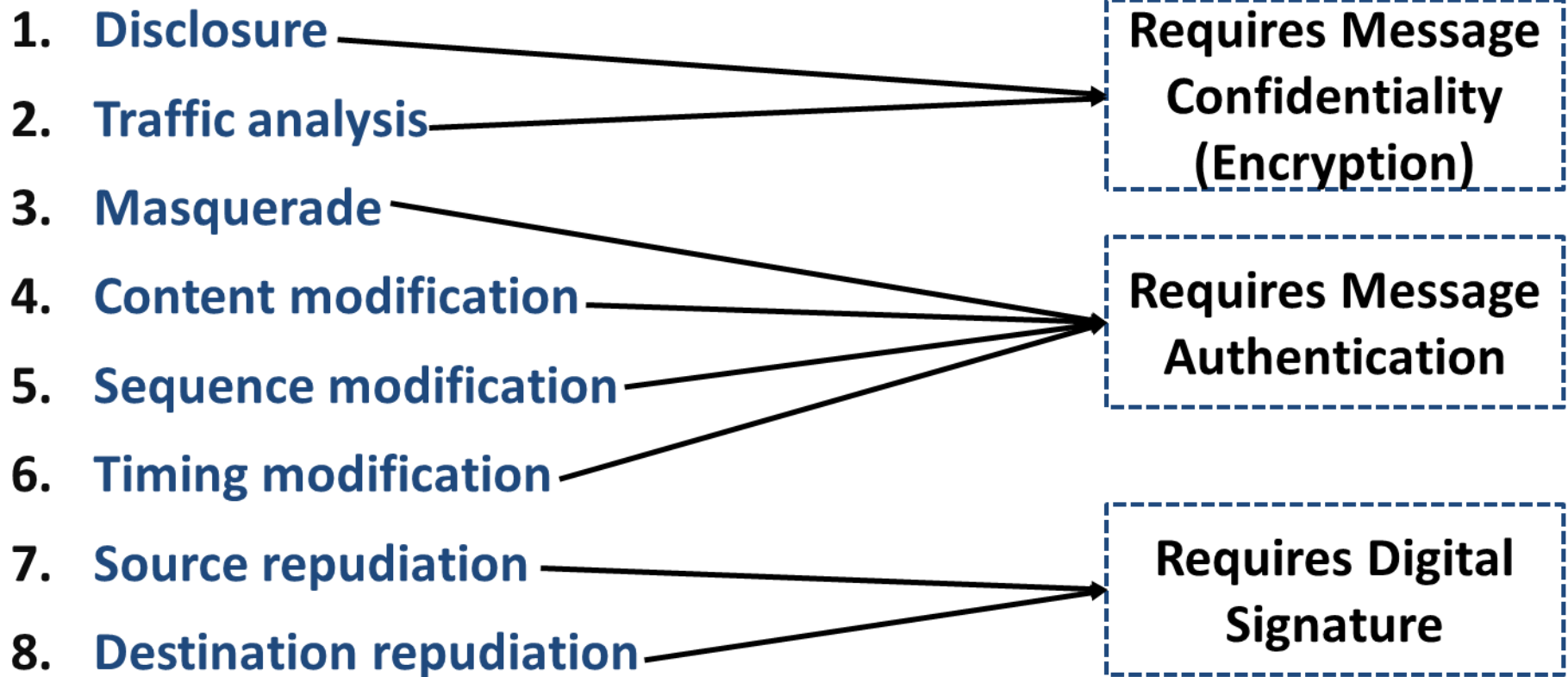
Requirement for Message Authentication Code

- Disclosure
- Traffic analysis
- Masquerade
- Content modification
- Sequence modification
- Timing modification
- Source repudiation
- Destination repudiation

Requirement for Message Authentication Code

- **Disclosure:** Release of message contents.
- **Traffic analysis:** Analyses the traffic and observe the pattern of traffic between parties.
- **Masquerade:** Insertion of messages into the network from a fraudulent source.
- **Content modification:** Changes to the contents of a message.
- **Sequence modification:** Any modification to a sequence of messages between parties.
- **Timing modification:** Delay or replay of messages.
- **Source repudiation:** Denial of transmission of message by source.
- **Destination repudiation:** Denial of receipt of message by destination.

Requirement for Message Authentication Code



Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>



Security of MACs

- **Goals:**
 - Unforgeability: Attacker can't produce a valid MAC for a new message.
 - Integrity: Detect any modifications.
- **Adversarial Model:**
 - Can see MACs for chosen messages.
 - Cannot guess or compute MACs for unqueried messages.

Security of MACs

- **Forgery Attacks:**
 - Existential Forgery: Attacker creates some valid message-MAC pair.
 - Selective Forgery: MAC for a specific chosen message.
- **Replay Attacks:** Resending old valid messages.
- **Length Extension Attacks:** Exploits certain hash-based MACs (e.g., $H(m || \text{pad} || x)$).

Security of MACs

- **CBC-MAC:** Secure for fixed-length messages
- **HMAC (Hash-based MAC):**
 - Uses SHA-256, SHA-3, etc.
 - Resistant to length extension.
- **CMAC:** AES-based alternative to CBC-MAC.
- **Poly1305:** High-speed MAC used with ChaCha20.

Security of MACs

- **Security Best Practices for Using MACs**
 - Use cryptographically secure primitives (e.g., HMAC-SHA256).
 - Do not reuse keys across protocols.
 - Use MAC with nonces or timestamps to prevent replay.
 - Authenticate then encrypt, or use AEAD (e.g., AES-GCM).
- **Summary:**
 - MACs ensure authenticity and integrity.
 - Secure design, proper key management, and resistant constructions are essential.

HMAC

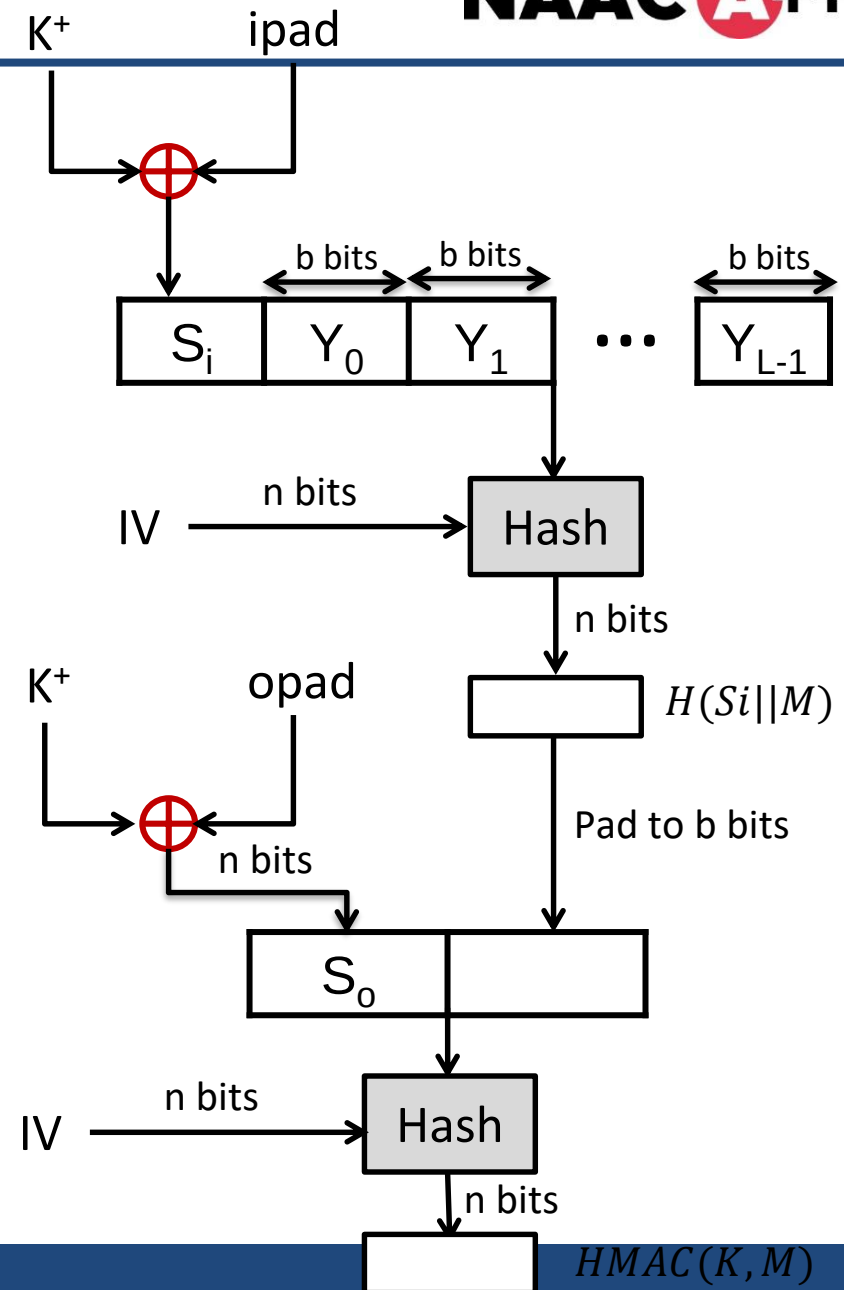
- Cryptographic hash functions such as MD5 and SHA generally execute faster in software than symmetric block ciphers such as DES.
- Library code for cryptographic hash functions is widely available.

HMAC

- To use available hash functions without modifications.
- To allow for easy replaceability of the embedded hash function in case faster or more secure hash functions are found or required.
- To preserve the original performance of the hash function without incurring a significant degradation..
- To use and handle keys in a simple way.
- To have a well understood cryptographic analysis of the strength of the authentication mechanism based on reasonable assumptions about the embedded hash function.

HMAC

1. Append zeros to the left end of K to create a b -bit string K^+
2. XOR K^+ with $ipad$ to produce the b -bit block S_i .
3. Append M to S_i .
4. Apply H to the stream generated in step 3.
5. XOR K^+ with $opad$ to produce the b -bit block S_0 .
6. Append the hash result from step 4 to S_0 .
7. Apply H to the stream generated in step 6 and output the result.



HMAC

- H = Embedded hash function (e.g. MD5, SHA-1, RIPEMD-160).
- IV = Initial value that is input to hash function.
- M = Message input to HMAC.
- Y_i = i th block of M .
- L = Number of blocks in M .
- N = Length of hash code produced by embedded hash function.
- $K +$ = K padded with zeros on the left so that the result is b bits in length.
- $ipad$ = 00110110 (36 in hexadecimal) repeated $b/8$ times.
- $opad$ = 01011100 (5C in hexadecimal) repeated $b/8$ times.

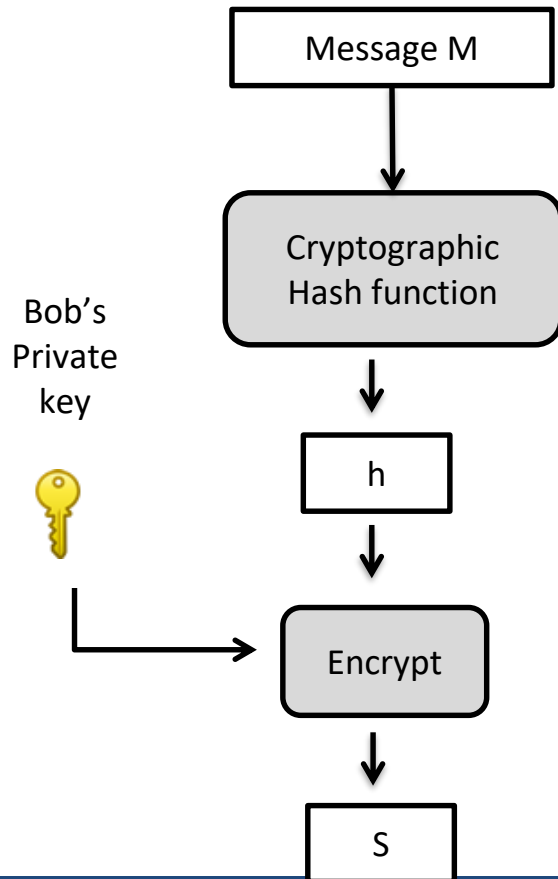
Digital Signature

- A digital signature is an authentication mechanism that enables the **creator of a message to attach a code that acts as a signature.**
- Typically the signature is formed by taking the **hash of the message** and encrypting the message with the **creator's private key.**
- The signature **guarantees the source and integrity of the message.**
- The digital signature standard (DSS) is an NIST standard that uses the secure hash algorithm (SHA).

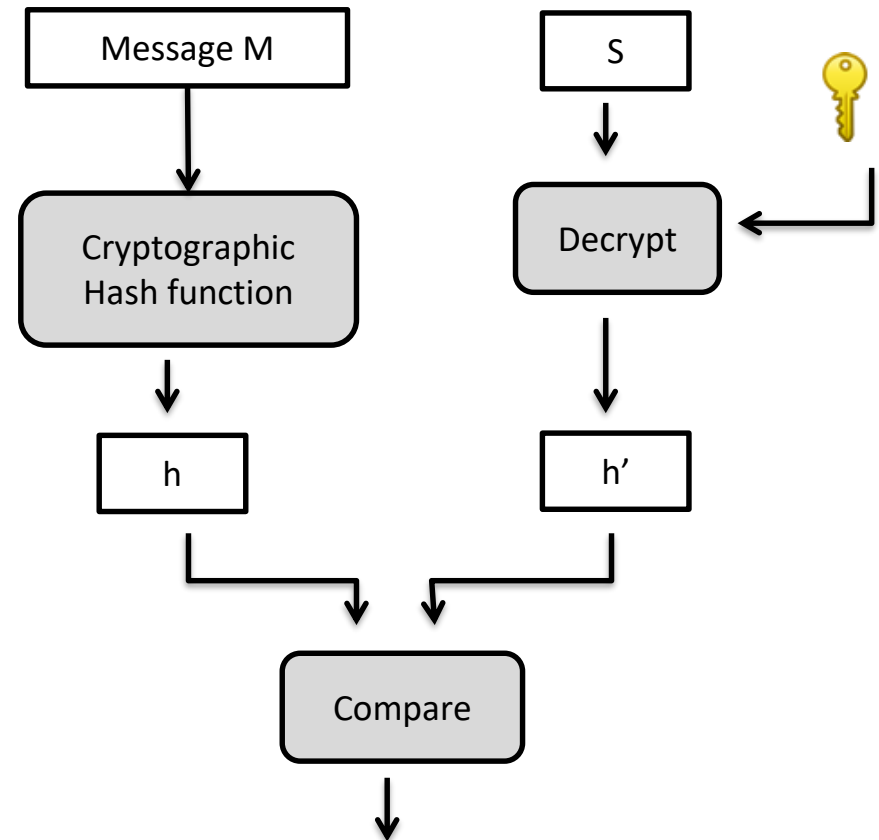
Digital Signature



Bob



Alice



Digital Signature

- **Hash Code**
- A hash of the message, if appended to the message itself, only protects against accidental changes to the message, as an attacker who modifies the message can simply calculate a new hash and use it instead of the original one. So this only gives integrity.
- **MAC**
- A message authentication code (MAC) (sometimes also known as keyed hash) protects against message forgery by anyone who doesn't know the secret.
- This means that the receiver can forge any message – thus we have both integrity and authentication (as long as the receiver doesn't have a split personality), but not non-repudiation.

Digital Signature

- **Digital Signature**
- A digital signature is created with a private key, and verified with the corresponding public key of an asymmetric key-pair.
- Only the holder of the private key can create this signature, and normally anyone knowing the public key can verify it. So digital signature provides non-repudiation.

Digital Signature - Attacks

- **C = Attacker, A = victim**
- **Key-only attack:** C only knows A's public key.
- **Known message attack:** C has set of messages, signatures.
- **Generic chosen message attack:** C obtains A's signatures on messages selected without knowledge of A's public key.
- **Directed chosen message attack:** C obtains A's signatures on messages selected after knowing A's public key.
- **Adaptive chosen message attack:** C may request signatures on messages depending upon previous message-signature pairs.

Digital Signature - Forgeries

- **Total break:** C determines A's private key.
- **Universal forgery:** C finds an efficient signing algorithm that provides an equivalent way of constructing signatures on arbitrary messages.
- **Selective forgery:** C forges a signature for a particular message chosen by C.
- **Existential forgery:** C forges a signature for a particular message not chosen by C. Consequently, this forgery may only be a minor nuisance to A.

Digital Signature - Requirements

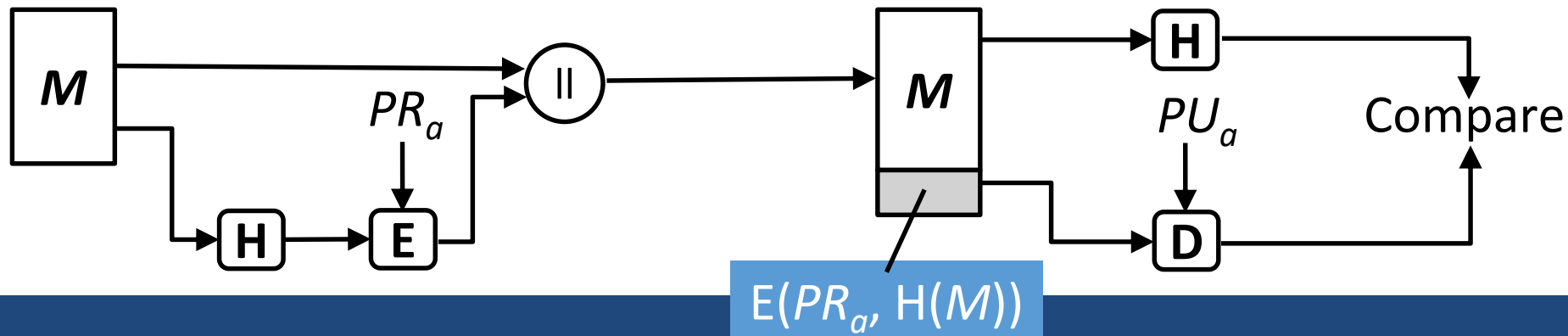
- The signature must be a **bit pattern** that depends on the message being signed.
- The signature must use some information **unique** to the sender to prevent both forgery and denial.
- It must be relatively **easy to produce** the digital signature.
- It must be relatively **easy to recognize** and **verify** the digital signature.
- It must be computationally **infeasible to forge** a digital signature, either **by constructing a new message for an existing digital signature** or **by constructing a fraudulent digital signature for a given message**.
- It must be practical to **retain a copy of the digital signature** in storage.

Digital Signature Standard

- The **DSS** uses an algorithm that is designed to provide only the digital signature function.
- Unlike RSA, it cannot be used for encryption or key exchange.

Digital Signature Standard – RSA Approach

- In the RSA approach, the message to be signed is input to a hash function that produces a **secure hash code of fixed length**.
- This hash code is then encrypted using the **sender's private key** to form the **signature**.
- Both the **message** and the **signature** are then **transmitted**.
- The **recipient** takes the message and **produces a hash code**.

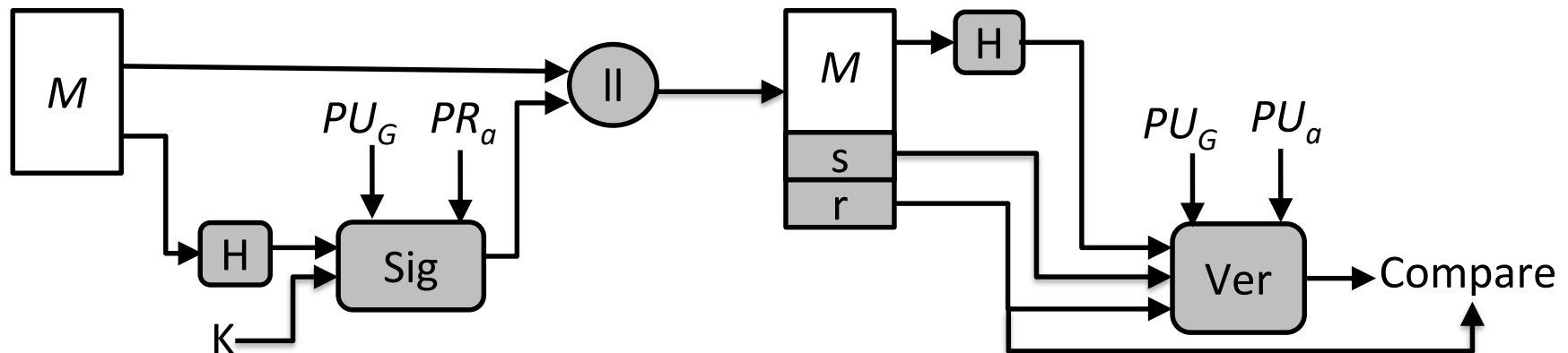


Digital Signature Standard - RSA Approach

- The recipient also **decrypts** the **signature** using the **sender's public key**.
- If the **calculated hash code matches** the **decrypted signature**, the signature is accepted as **valid**.
- Because only the sender knows the private key, only the sender could have produced a valid signature.

Digital Signature Standard – DSA Approach

- The **hash code** is provided as **input to a signature** function along with a random number **K** generated for this particular signature.
- The signature function also depends on the **sender's private key** (PR_a) and a set of parameters known to a group of communicating principals.
- We can consider this set to constitute a **global public key (PUG)**
- The **result is a signature** consisting of **two components**, labelled as **s** and **r**.



Digital Signature Standard

- At the receiving end, the **hash code** of the incoming message is generated.
- This plus the **signature** is input to a **verification function**.
- The verification function also depends on the global public key(PUG) as well as the sender's public key (PUa), which is paired with the sender's private key.
- The **output** of the **verification function** is a **value** that is **equal to the signature component r if the signature is valid**.
- The signature function is such that only the sender, with knowledge of the private key, could have produced the valid signature.

Digital Signature Algorithm

Global Public-Key Components

- p : Prime number where $2^{L-1} < p < 2^L$
For $512 \leq L \leq 1024$ and L a multiple of 64;
i.e., bit length of between 512 and 1024 bits in
increments of 64 bits.
- q : Prime divisor of $(p - 1)$, where $2^{N-1} < p < 2^N$
i.e., bit length of N bits.
- g : $= h^{(p-1)/q} \bmod p$
Where h is any integer with $1 < h < (p - 1)$
Such that $h^{(p-1)/q} \bmod p > 1$

Digital Signature Algorithm

User's Private key

x Random or pseudorandom integer with $0 < x < q$

User's Public key

$$y = g^x \bmod p$$

User's Per-Message Secret Number

k Random or pseudorandom integer with $0 < k < q$

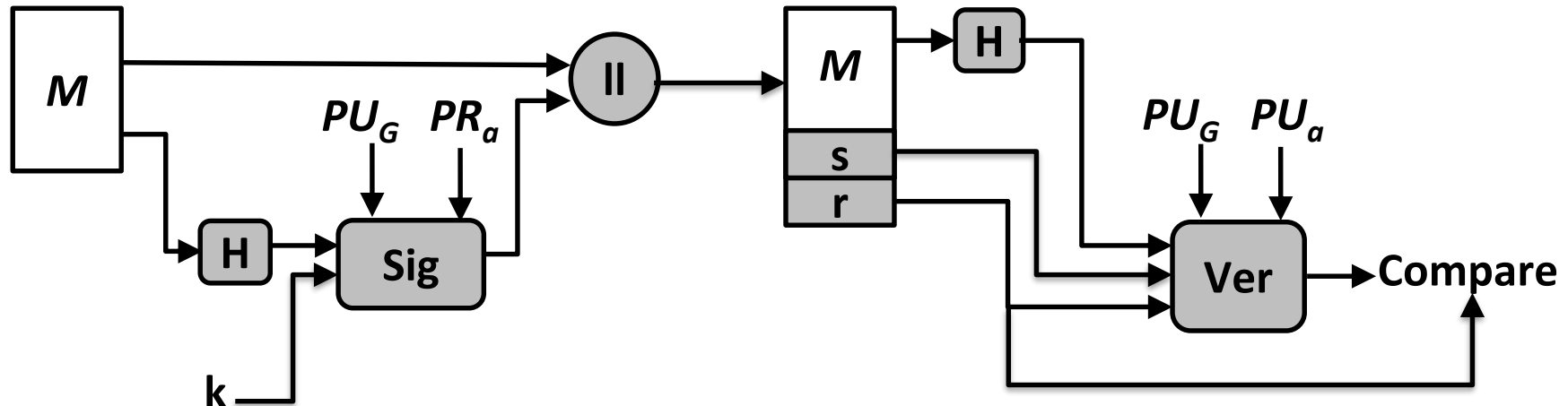
Digital Signature Algorithm

Signing

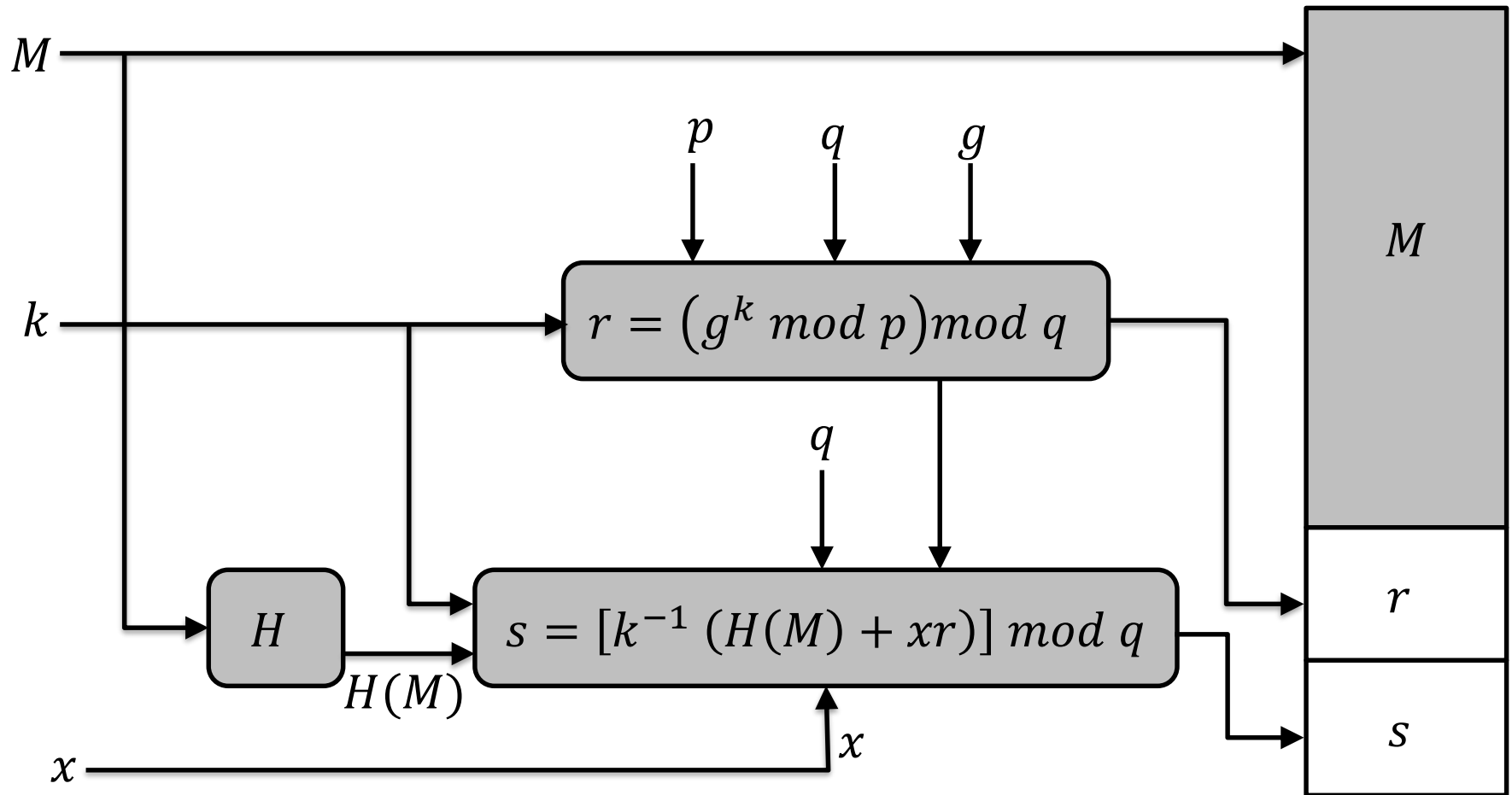
$$r = (g^k \bmod p) \bmod q$$

$$s = [k^{-1} (H(M) + xr)] \bmod q$$

$$\text{Signature} = (r, s)$$



Digital Signature Algorithm



Digital Signature Algorithm

Verifying

$$w = (s')^{-1} \bmod q$$

$$u_1 = [H(M')w] \bmod q$$

$$u_2 = (r')w \bmod q$$

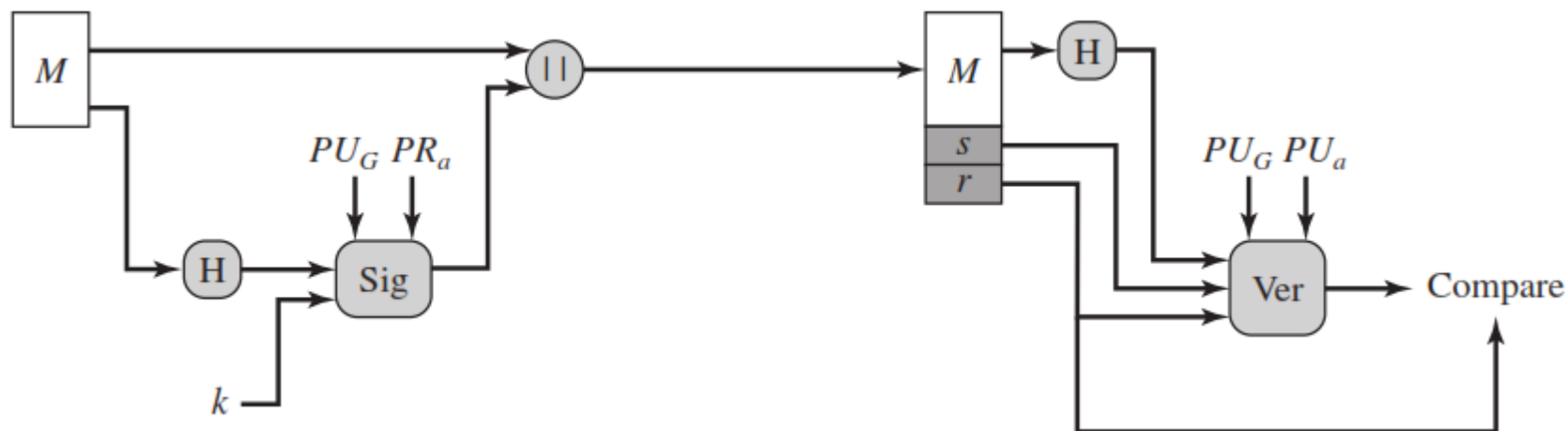
$$v = [(g^{u_1} y^{u_2}) \bmod p] \bmod q$$

$$\text{TEST: } v = r'$$

M = message to be signed

$H(M)$ = hash of M using SHA-1

M', r', s' = received versions of M, r, s



Digital Signature Algorithm

Verifying

$$w = (s')^{-1} \bmod q$$

$$u_1 = [H(M')w] \bmod q$$

$$u_2 = (r')w \bmod q$$

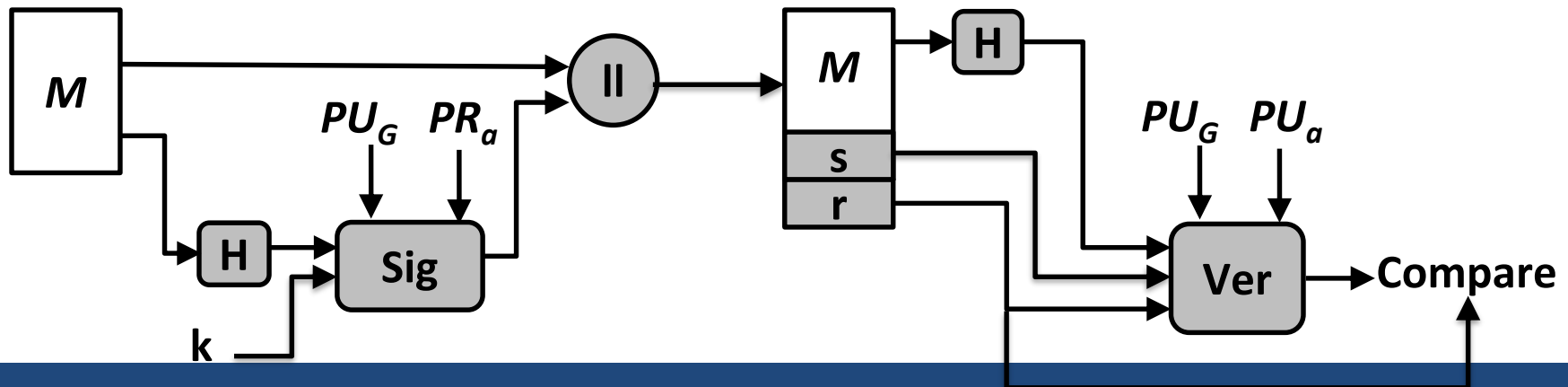
$$v = [(g^{u_1} y^{u_2}) \bmod p] \bmod q$$

$$\text{Test: } v = r'$$

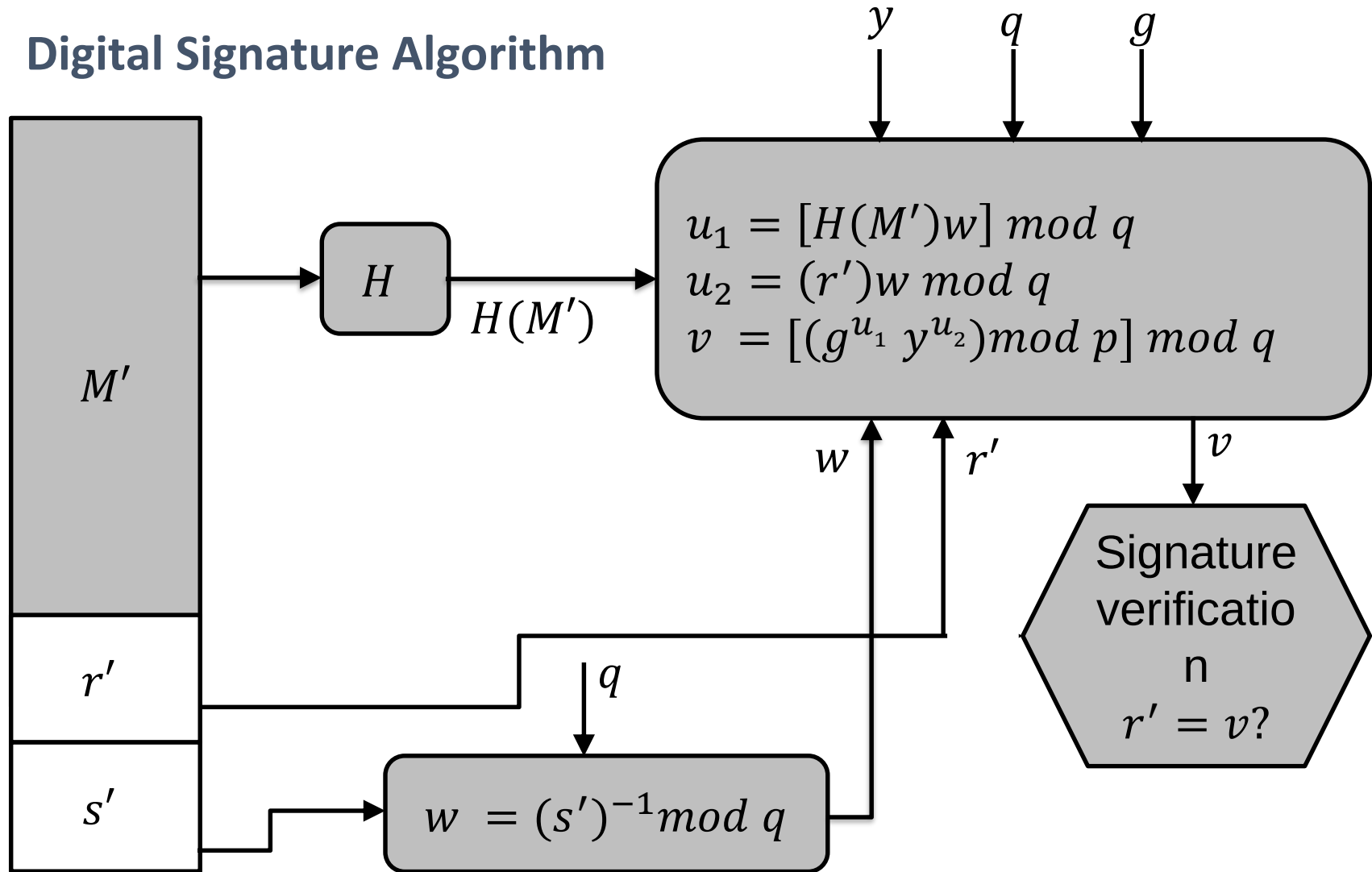
M = message to be signed

$H(M)$ = hash of M using SHA-1

M', r', s' = received versions of M, r, s



Digital Signature Algorithm



Digital Signature Algorithm

- Uses private key for encryption (signing)
- Uses public key for decryption (verification)
- Each user generates their key
 - Chooses a secret key (number): $1 < X_A < q$
 - Compute their public key: $Y_A = a^{(X_A)} \bmod q$

Digital Signature Algorithm – ElGamal Approach

- Alice signs a message M to Bob by computing
 - The hash $m=H(M)$, $0 \leq m \leq (q-1)$
 - Choose random integer K with $1 \leq K \leq (q-1)$ and $\gcd(K, q-1)=1$
 - Compute temporary key: $S1=a^K \mod q$
 - Compute K^{-1} the inverse of $K \mod (q-1)$
 - Compute the value: $S_2=K^{-1} (m-X_A S1) \mod (q-1)$
 - Signature is: $(S1, S2)$
- Any user B can verify the signature by computing

$$V1=a^m \mod q$$

$$V_2=[Y_A]^{S1} [S_1]^{S2} \mod q$$

$$= [((a)^{X_A})]^{S1} [((a)^K)]^{S2} \mod q$$

$$= a^{(X_A S1+m-X_A S1)} \mod q = a^m \mod q$$
- Signature is valid if $V1=V2$

Digital Signature Algorithm – ElGamal Approach

- Use field GF(19) $q=19$ and $a=10$
- Alice computes her key:
- A chooses $X_A=16$ & computes $Y_A=10^{16} \bmod 19 = 4$
- Alice signs message with hash $m=14$ as (3, 4)
- Choosing random $K=5$ which has $\gcd(18, 5)=1$
- Computing $S1=10^5 \bmod 19 = 3$
- Finding $K^{(-1)} \bmod (q-1)=5^{(-1)} \bmod 18=11$
- Computing $S2=11(14-16*3) \bmod 18=4$
- Any user B can verify the signature by computing
- $V1=10^{14} \bmod 19=16$
- $V2=4^3*3^4=5184=16 \bmod 19$
- Since $16=16$ signature is valid

Digital Signature Algorithm – Schnorr Algorithm

- Also uses exponentiation in a finite (Galois)
 - Security based on discrete logarithms
- Minimizes message dependent computation
 - Multiplying a **$2n$ bit** integer with an **n bit** integer
- Main work can be done in idle time
- Have using a prime modulus p
 - $p-1$ has a prime factor q of appropriate size
 - Typically p is **1024-bit** and q is **160-bit** numbers

Digital Signature Algorithm - Schnorr Algorithm s

- Choose suitable primes p and q
- choose a such that $a^q = 1 \bmod p$
- (a, p, q) are global parameters for all
- Each user generates a key
- Chooses a secret key (number): $0 < S_A < q$
- Compute their public key: $V_A = a^{(-S_A)} \bmod q$

Digital Signature Standard

- User signs message by
- Choosing random number r with $0 < r < q$ and computing: $X = a^r \mod p$
- Concatenate message with X and hash result to computing: $e = H(M || X)$
- Computing: $Y = (r + se) \mod q$
- Signature pair: (e, Y)
- Any other user can verify the signature as follows:
- Computing:
$$\begin{aligned} [X]^{a'} &= a^Y V^{e'} \mod p = a^Y a^{(-Se)} \mod p \\ &= a^{(Y-se)} \mod p = a^r \mod p = X \end{aligned}$$
- Verifying that: $e = H(M || X')$

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>

